

Exploring Multi-Armed Bandits in Recommendation Systems

A Comparative Study

Manish [24n0451] Riya [24n0452] Kishan [24n0453] Ditipriya [23n0465]

Guide: Prof. Manjesh Kumar Hanawal

May 5, 2025

Outline

- 1 Introduction
- 2 Literature Review
- 3 Problem Formulation
- 4 Datasets
- 5 Algorithms
- 6 Evaluation
- 7 Conclusion

Problem Statement

- Modern recommendation systems must adapt in real time to:
 - Individual users' preferences
 - Evolving inventories
 - Shifting behavioral patterns
- Traditional offline models struggle with:
 - “Cold-start” problem
 - Nonstationary user interests
 - Need to balance exploration vs. exploitation
- Our approach: Formulate product recommendation as a **non contextual and contextual multi-armed bandit** problem

Our Project Scope

- **Goal:** Compare and evaluate bandit algorithms for recommendation systems
- **Datasets:**
 - MovieLens 1M (50 most-rated movies, 109,804 interactions)
 - Zozo/men (33 men's fashion items, 440,435 impressions)
- **Metrics:**
 - Cumulative reward
- Using the replay method for unbiased offline evaluation

Xiang (2021) - E-commerce Applications

- Evaluated ϵ -greedy, Thompson Sampling, UCB1
- Under non-stationary and delayed-feedback conditions
- Introduced batch-update framework
- Live A/B test: **6.13%** CTR and **16.1%** CVR improvements

Gangan (2021) - Survey of MAB Algorithms

- Overview of bandit methods: ϵ -greedy, UCB, Thompson Sampling, EXP3
- Contextual extensions and advanced variants
- Industry case studies: Spotify, Netflix, Yahoo
- Importance of parameter tuning

Practical "Productization" Challenges

- **Context (feature) engineering**
 - Which features to encode?
 - How to encode user/item features?
 - Poor feature selection degrades performance
- **Dynamic action (arm) management**
 - Adding new items (arms) on the fly
 - Seamless incorporation without restarting learners

Our focus: Explore and compare various MAB approaches

Multi-Armed Bandit Formulation

For each interaction round $t = 1, \dots, T$:

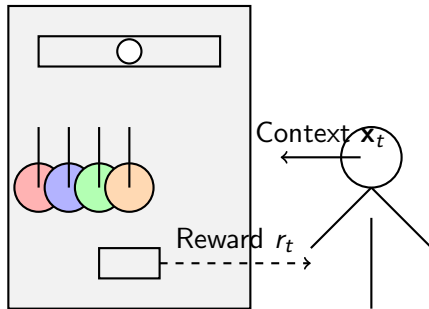
- 1 Observe context $\mathbf{x}_t \in \mathbb{R}^d$ (user/item features)
- 2 Select arm $a_t \in \{1, \dots, K\}$ (recommend an item)
- 3 Receive reward $r_{t,a_t} \in [0, 1]$ (click or engagement)

Objective: Learn policy $\pi(a \mid \mathbf{x})$ that maximizes cumulative reward

$$R_T = \sum_{t=1}^T r_{t,a_t}$$

or equivalently minimizes regret

$$\mathcal{R}_T = \sum_{t=1}^T (r_t^* - r_{t,a_t})$$



MovieLens 1M

- 1,000,209 user-movie ratings
- 6,040 unique users
- 3,706 unique movies
- User demographics (gender, age, occupation)
- Movie metadata (title, genres)
- Used 50 most-rated movies, 109,804 interactions

Zozo Men's Fashion

- 440,435 impression events
- 34 men's fashion items
- Features:
 - User features (4 dimensions)
 - Item features (4 dimensions)
 - User-item affinity scores
 - Position, click, propensity score

Contextual Bandits

- **LinUCB:**
 - Ridge-regularized linear model
 - Upper confidence bound exploration
- **ϵ -Greedy (Ridge):**
 - Ridge regression reward estimation
 - Random exploration with probability ϵ
- **ϵ -Greedy (Logistic):**
 - Logistic regression for binary rewards
 - Online gradient updates
- **Thompson Sampling:**
 - Bayesian approach with online logistic regression
 - Laplace approximation for posterior

Non-Contextual Bandits

- **UCB1:**
 - Confidence bound based on play count
 - Balance exploration and exploitation
- **ϵ -Greedy:**
 - Simple average reward estimation
 - Random exploration with probability ϵ
- **Thompson Sampling:**
 - Beta-Bernoulli model
 - Posterior sampling for exploration
- **Random Policy:**
 - Uniform random selection
 - Baseline for comparison

Random Policy Algorithm

Algorithm 1 Random Policy

Require: Number of arms K , total rounds T

- 1: **for** $t = 1, \dots, T$ **do**
 - 2: Select arm $a_t \sim \text{Uniform}\{1, \dots, K\}$ {Uniform random selection}
 - 3: Receive reward r_t
 - 4: **end for**
-

- Simplest possible policy - no adaptation
- Used as baseline for comparison
- Pure exploration (no exploitation)
- Expected regret grows linearly with time

ϵ -Greedy Algorithm (Context-Free)

Algorithm 2 ϵ -Greedy (Context-Free)

Require: Exploration rate $\epsilon \in [0, 1]$, number of arms K , total rounds T

- 1: Initialize counts $N(a) \leftarrow 0$ and estimates $Q(a) \leftarrow 0$ for all $a = 1, \dots, K$
 - 2: **for** $t = 1, \dots, T$ **do**
 - 3: Draw $u \sim \text{Uniform}(0, 1)$
 - 4: **if** $u < \epsilon$ **then**
 - 5: Select arm $a_t \sim \text{Uniform}\{1, \dots, K\}$ {Explore}
 - 6: **else**
 - 7: Select arm $a_t \leftarrow \arg \max_a Q(a)$ {Exploit}
 - 8: **end if**
 - 9: Receive reward $r_t \in [0, 1]$
 - 10: $N(a_t) \leftarrow N(a_t) + 1$ {Update count}
 - 11: $Q(a_t) \leftarrow Q(a_t) + \frac{1}{N(a_t)}(r_t - Q(a_t))$ {Update average}
 - 12: **end for**
-

UCB1 Algorithm (Context-Free)

Algorithm 3 UCB1 (Context-Free)

Require: Number of arms K , total rounds T

- 1: Initialize counts $N(a) \leftarrow 0$, value estimates $Q(a) \leftarrow 0$ for $a = 1, \dots, K$
 - 2: **for** $a = 1, \dots, K$ **do**
 - 3: Play arm a , receive reward r
 - 4: $N(a) \leftarrow 1$, $Q(a) \leftarrow r$ {Initial plays}
 - 5: **end for**
 - 6: **for** $t = K + 1, \dots, T$ **do**
 - 7: **for** $a = 1, \dots, K$ **do**
 - 8: $U(a) \leftarrow Q(a) + \alpha \sqrt{\frac{2 \ln t}{N(a)}}$ {UCB score}
 - 9: **end for**
 - 10: Select arm $a_t \leftarrow \arg \max_a U(a)$
 - 11: Receive reward $r_t \in [0, 1]$
 - 12: $N(a_t) \leftarrow N(a_t) + 1$ {Update count}
 - 13: $Q(a_t) \leftarrow Q(a_t) + \frac{1}{N(a_t)} (r_t - Q(a_t))$ {Update average}
 - 14: **end for**
-

Thompson Sampling Algorithm (Context-Free)

Algorithm 4 Thompson Sampling (Context-Free)

Require: Number of arms K , total rounds T

- 1: Initialize successes $S(a) \leftarrow 0$ and failures $F(a) \leftarrow 0$ for all $a = 1, \dots, K$
- 2: **for** $t = 1, \dots, T$ **do**
- 3: **for** $a = 1, \dots, K$ **do**
- 4: Sample $\theta_a \sim \text{Beta}(S(a) + 1, F(a) + 1)$ {Posterior sample}
- 5: **end for**
- 6: Select arm $a_t \leftarrow \arg \max_a \theta_a$
- 7: Receive binary reward $r_t \in \{0, 1\}$
- 8: **if** $r_t = 1$ **then**
- 9: $S(a_t) \leftarrow S(a_t) + 1$ {Update success count}
- 10: **else**
- 11: $F(a_t) \leftarrow F(a_t) + 1$ {Update failure count}
- 12: **end if**
- 13: **end for**

Algorithm 5 LinUCB with Disjoint Linear Models

Require: Exploration parameter $\alpha > 0$, number of arms K , feature dimension d

```
1: for  $a = 1$  to  $K$  do
2:    $A_a \leftarrow I_d$  {Initialize precision matrix}
3:    $b_a \leftarrow \mathbf{0}_d$  {Initialize response vector}
4: end for
5: for  $t = 1, 2, \dots, T$  do
6:   Observe feature vectors  $\{x_{a,t} \in \mathbb{R}^d\}_{a=1}^K$ 
7:   for each arm  $a = 1, \dots, K$  do
8:      $\hat{\theta}_a \leftarrow A_a^{-1} b_a$  {Ridge estimate}
9:      $p_{a,t} \leftarrow x_{a,t}^\top \hat{\theta}_a + \alpha \sqrt{x_{a,t}^\top A_a^{-1} x_{a,t}}$  {UCB score}
10:  end for
11:  Select  $a_t \leftarrow \arg \max_a p_{a,t}$ 
12:  Receive reward  $r_t \in [0, 1]$ 
13:   $A_{a_t} \leftarrow A_{a_t} + x_{a_t,t} x_{a_t,t}^\top$  {Update precision}
14:   $b_{a_t} \leftarrow b_{a_t} + r_t x_{a_t,t}$  {Update response}
15: end for
```

Algorithm 6 ϵ -Greedy with Ridge Regression

Require: Exploration rate $\epsilon \in [0, 1]$, number of arms K , dimension d , regularization $\lambda > 0$

```
1: for  $a = 1, \dots, K$  do
2:    $A_a \leftarrow \lambda I_d$   $\{d \times d \text{ ridge covariance matrix}\}$ 
3:    $b_a \leftarrow \mathbf{0}_d$   $\{d\text{-dim. response vector}\}$ 
4: end for
5: for  $t = 1, \dots, T$  do
6:   Observe feature vectors  $\{x_{a,t} \in \mathbb{R}^d\}_{a=1}^K$ 
7:   Draw  $u \sim \text{Uniform}(0, 1)$ 
8:   if  $u < \epsilon$  then
9:     Choose arm  $a_t \sim \text{Uniform}\{1, \dots, K\}$   $\{\text{Explore}\}$ 
10:  else
11:    for  $a = 1, \dots, K$  do
12:       $\hat{r}_a \leftarrow x_{a,t}^\top A_a^{-1} b_a$   $\{\text{Ridge prediction}\}$ 
13:    end for
14:    Select arm  $a_t \leftarrow \arg \max_a \hat{r}_a$   $\{\text{Exploit}\}$ 
15:  end if
16:  Receive reward  $r_t \in [0, 1]$ 
17:   $A_{a_t} \leftarrow A_{a_t} + x_{a_t,t} x_{a_t,t}^\top$ 
18:   $b_{a_t} \leftarrow b_{a_t} + r_t x_{a_t,t}$ 
19: end for
```

ϵ -Greedy with Logistic Regression

Algorithm 7 ϵ -Greedy with Logistic Regression

Require: Exploration rate $\epsilon \in [0, 1]$, number of arms K , dimension d , learning rate $\eta > 0$

```
1: for  $a = 1, \dots, K$  do
2:    $\theta_a \leftarrow \mathbf{0}_d$  {Initialize logistic weights}
3: end for
4: for  $t = 1, \dots, T$  do
5:   Observe feature vectors  $\{x_{a,t} \in \mathbb{R}^d\}_{a=1}^K$ 
6:   Draw  $u \sim \text{Uniform}(0, 1)$ 
7:   if  $u < \epsilon$  then
8:     Select arm  $a_t \sim \text{Uniform}\{1, \dots, K\}$  {Explore}
9:   else
10:    for  $a = 1, \dots, K$  do
11:       $\hat{p}_a \leftarrow \sigma(x_{a,t}^\top \theta_a)$   $\{\sigma(z) = 1/(1 + e^{-z})\}$ 
12:    end for
13:    Select arm  $a_t \leftarrow \arg \max_a \hat{p}_a$  {Exploit}
14:  end if
15:  Receive binary reward  $r_t \in \{0, 1\}$ 
16:   $\theta_{a_t} \leftarrow \theta_{a_t} + \eta (r_t - \sigma(x_{a_t,t}^\top \theta_{a_t})) x_{a_t,t}$  {Online update}
17: end for
```


Thompson Sampling with Online Logistic Regression

Algorithm 8 Thompson Sampling with Online Logistic Regression

Require: Prior precision $\lambda > 0$, number of arms K , feature dimension d

```
1: for  $a = 1, \dots, K$  do
2:    $A_a \leftarrow \lambda I_d$  {Posterior precision}
3:    $b_a \leftarrow \mathbf{0}_d$  {Precision-times-mean}
4:    $m_a \leftarrow A_a^{-1} b_a$  {Posterior mean}
5: end for
6: for  $t = 1, \dots, T$  do
7:   Observe context vectors  $\{x_{a,t} \in \mathbb{R}^d\}_{a=1}^K$ 
8:   for  $a = 1, \dots, K$  do
9:     Draw sample  $\theta_a \sim \mathcal{N}(m_a, A_a^{-1})$  {Posterior sample}
10:    Compute score  $p_{a,t} \leftarrow \sigma(x_{a,t}^\top \theta_a)$  {Sigmoid}
11:  end for
12:  Select arm  $a_t \leftarrow \arg \max_a p_{a,t}$ 
13:  Receive binary reward  $r_t \in \{0, 1\}$ 
14:  // Laplace update (one-step)
15:   $\tilde{p} \leftarrow \sigma(x_{a_t,t}^\top m_{a_t})$  {Current prediction}
16:   $s \leftarrow \tilde{p}(1 - \tilde{p})$  {Fisher information}
17:   $A_{a_t} \leftarrow A_{a_t} + s x_{a_t,t} x_{a_t,t}^\top$  {Update precision}
18:   $b_{a_t} \leftarrow b_{a_t} + (r_t - \tilde{p}) x_{a_t,t}$  {Update sufficient statistic}
19:   $m_{a_t} \leftarrow A_{a_t}^{-1} b_{a_t}$  {Update posterior mean}
```

Offline Evaluation using Replay Method

- Process historical logs in chronological order
- Only evaluate actions matching logged policy

Performance Metrics

- **Cumulative Reward:** Total clicks/conversions over time
- **Runtime:** Computational efficiency for real-time systems

Data Preprocessing Summary

Step 1: Define binary Reward: 1 if Rating = 5, else 0

Step 2: Create AgeGroup buckets via `pd.cut`

Step 3: Select top N=50 movies by number of ratings

Step 4: Extract genres, create one-hot encoded columns

Step 5: Merge ratings with user and movie features by IDs

Step 6: One-hot encode categorical user features

Step 7: Build feature matrix X by concatenating encodings

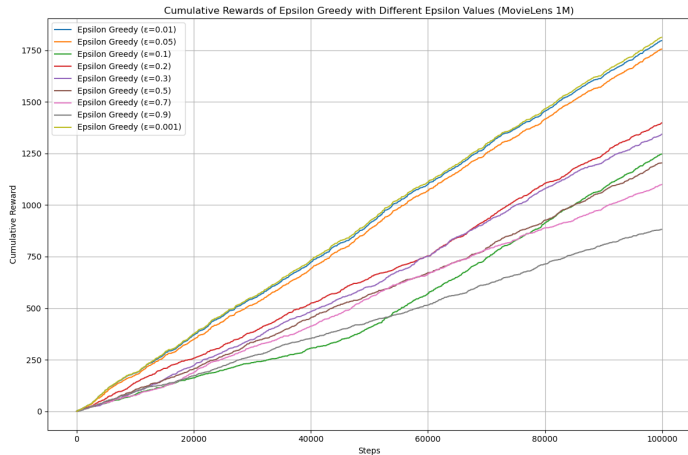
Step 8: Map movies to "arms" with indices $\{0, \dots, 49\}$

Step 9: Final dataset: UserID, MovieID, ArmIndex, Reward, Timestamp

- **Final Dataset shape:** (109804, 5)
- **Feature matrix:** $X \in \mathbb{R}^{n \times d}$ where d = dimension of features (109804, 47)

Parameter Tuning: ϵ -Greedy Strategy

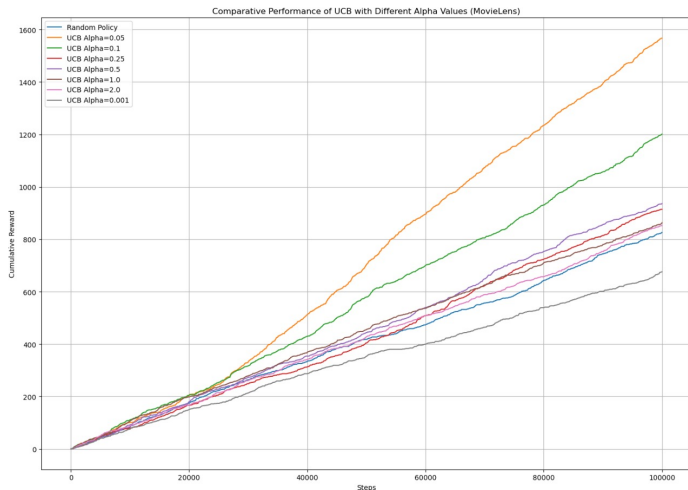
- Best performance with smaller ϵ values (0.001-0.01)
- Higher exploration ($\epsilon \geq 0.5$) yields significantly lower rewards
- Nearly 1800 cumulative reward at 100K steps with optimal setting



Analysis: Lower ϵ values perform better in this dataset, suggesting that exploitation is more valuable than exploration after initial learning.

Parameter Tuning: UCB

- UCB with $\alpha = 0.05$ dramatically outperforms all other configurations
- Nearly 2x reward vs. random policy at 100K steps
- Higher α values (>0.1) perform worse

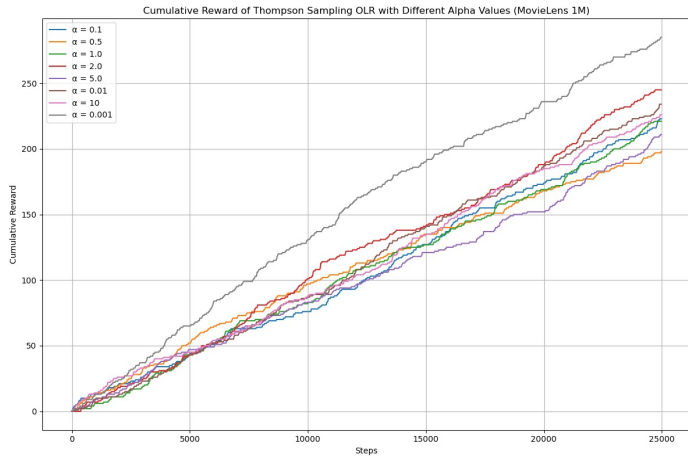


Analysis: UCB with carefully tuned confidence parameter significantly outperforms other approaches, suggesting it effectively balances exploration-exploitation for this recommendation task.

Parameter Tuning: Thompson Sampling

Thompson Sampling:

- $\alpha = 0.001$ significantly outperforms others
- Low exploration parameter works best

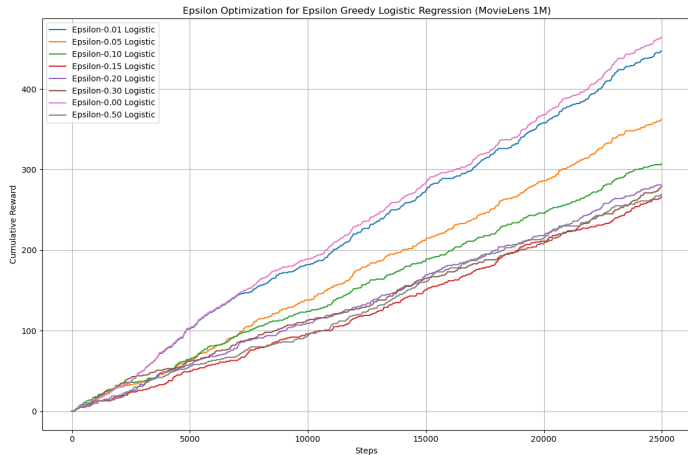


Analysis: Both algorithms show that minimal uncertainty/exploration parameters lead to higher rewards in the MovieLens dataset.

Parameter Tuning: ϵ Greedy with Logistic Regressions

Logistic Regression:

- $\epsilon = 0.0001$ outperforms all other values
- Almost pure exploitation strategy works best once model is trained

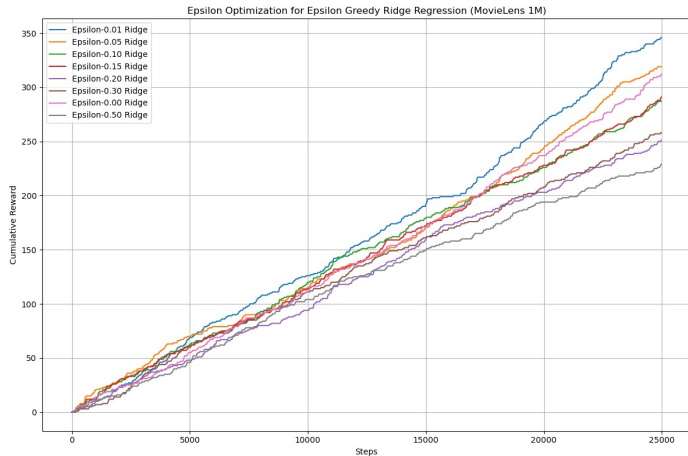


Analysis: Contextual bandits with regression benefit from very low exploration rates, with logistic regression favoring nearly pure exploitation.

Parameter Tuning: ϵ Greedy with ridge Regressions

Ridge Regression:

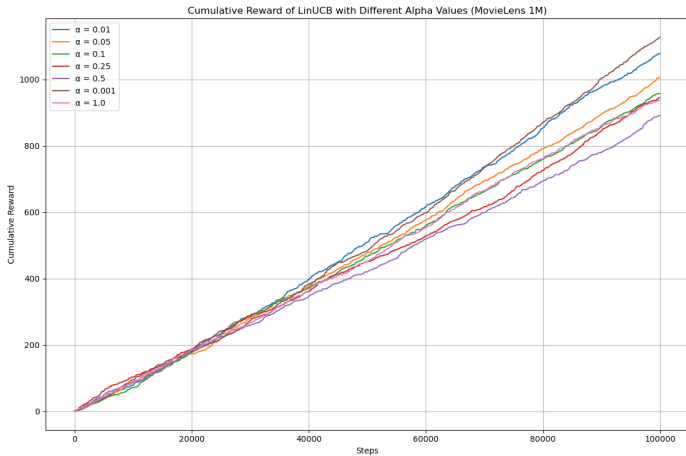
- $\epsilon = 0.01$ performs best
- Small exploration helps improve model accuracy



Parameter Tuning: LinUCB

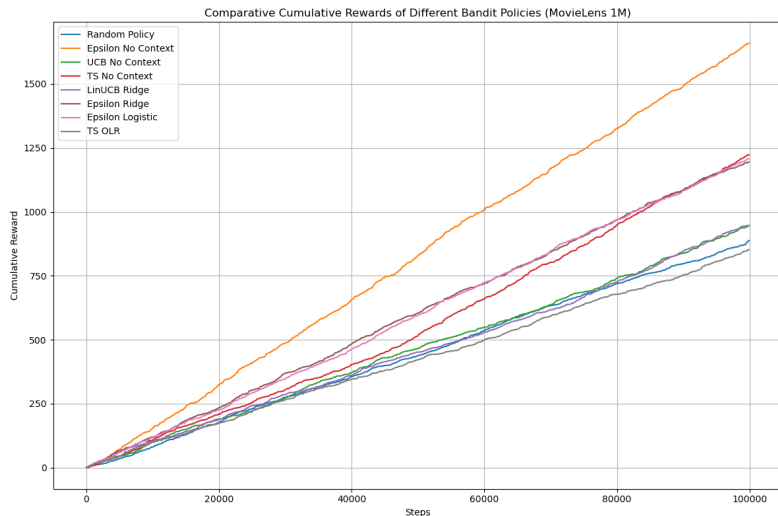
LinUCB:

- $\alpha = 0.001$ provides best performance
- Too much exploration ($\alpha > 0.5$) reduces rewards



Analysis: Both algorithms show that minimal uncertainty/exploration parameters lead to higher rewards in the MovieLens dataset.

Final Comparison of Bandit Policies



Policy	Reward
Random Policy	890.0
Epsilon No Context	1660.0
UCB No Context	948.0
TS No Context	1224.0
LinUCB Ridge	946.0
Epsilon Ridge	1196.0
Epsilon Logistic	1210.0
TS OLR	853.0

Table: Final rewards at 100K steps

Key Findings

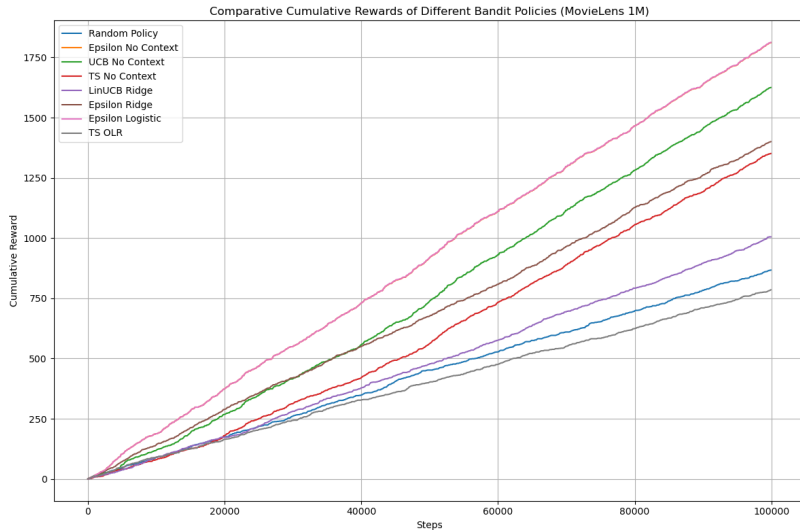
Key Findings

- Context-free Epsilon-Greedy with optimized ϵ significantly outperforms all alternatives, achieving **86.5%** higher reward than a random policy.
- Contextual approaches (LinUCB, Epsilon Ridge) show no advantage over simpler methods for this movie recommendation task.
- Thompson Sampling without context (TS No Context) performs well but still lags behind optimized Epsilon-Greedy.

Future Work

- Address feature engineering challenges
- Develop dynamic arm management strategies
- Explore hybrid models combining bandits with collaborative filtering

Comparative Cumulative Rewards of Different Bandit Policies Fine Tuned



Analysis of tuned Bandit Algorithm Performance

Key Observations:

• Performance Ranking:

- Epsilon Logistic (pink) shows superior performance (1800 reward)
- LinUCB Ridge (green) ranks second (1600 reward)
- Epsilon Ridge and UCB No Context show strong middle performance
- Random Policy and TS OLR demonstrate weakest performance

• Algorithmic Insights:

- Context-aware methods consistently outperform context-free approaches
- All algorithms perform similarly until 20,000 steps (exploration phase)
- Performance divergence accelerates after exploration phase
- Logistic regression models show steeper reward curves

Performance Analysis: Random Policy (Baseline)

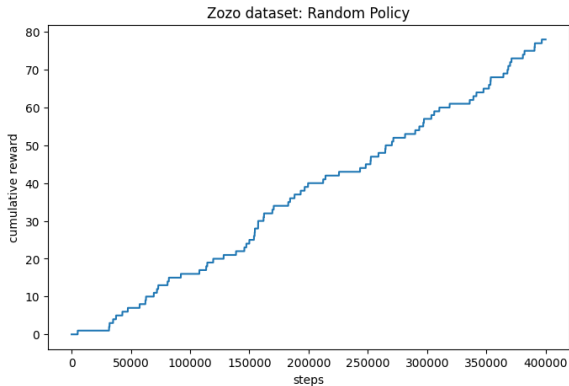


Figure: Random Policy (Baseline)

- Pure exploration strategy (baseline)
- No exploitation of knowledge
- Achieves only ≈ 80 reward after 400K steps
- Demonstrates the value of strategic algorithms

Performance Analysis: Epsilon-Greedy Strategy

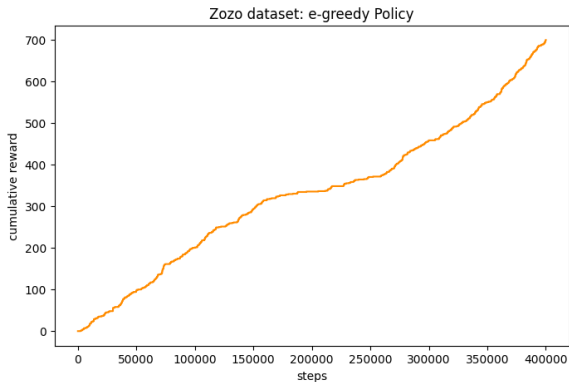


Figure: Epsilon-Greedy ($\varepsilon = 0.01$)

- Balances exploration (random) and exploitation (best known)
- Explores with probability $\varepsilon = 0.01$
- Achieves ≈ 690 reward after 400K steps
- Near-linear growth with some plateaus

Performance Analysis: UCB Algorithm

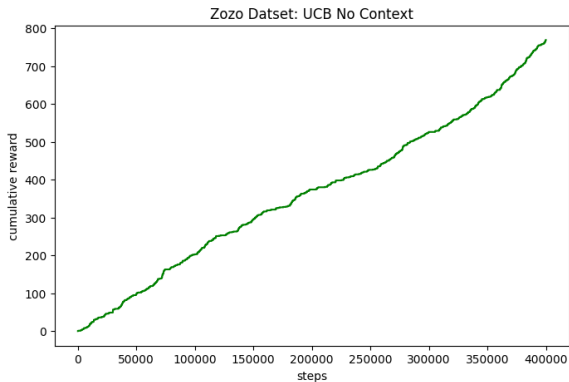


Figure: UCB Algorithm (No Context)

- Best performing algorithm
- Uses confidence bounds to balance exploration/exploitation
- Achieves ≈ 760 reward after 400K steps
- Consistent growth throughout experiment

Performance Analysis: Thompson Sampling

Zozo Dataset: Thompson Without Context

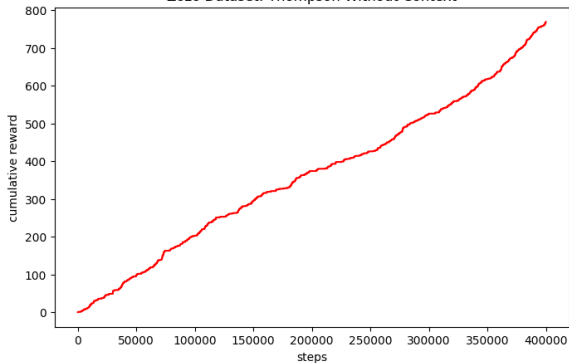
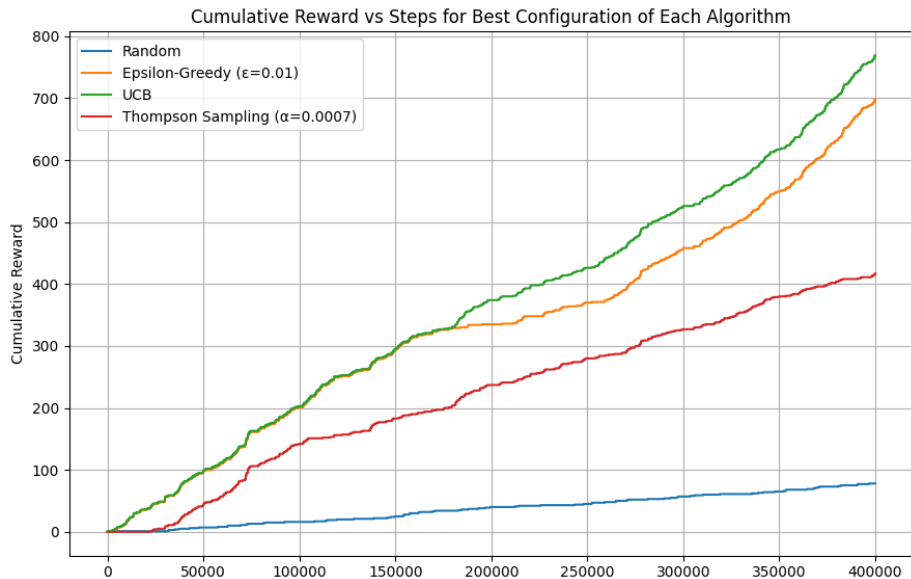


Figure: Thompson Sampling

- Bayesian approach to exploration/exploitation
- Uses probability matching for arm selection
- Achieves ≈ 760 reward after 400K steps
- Shows steady improvement over time

Multi-Armed Bandit Algorithms: Comparative Analysis



Summary: Multi-Armed Bandit Algorithms on Zozo Dataset

Algorithm	Final Reward	Rank	Key Characteristic
UCB	≈ 760	1	Confidence-based exploration
Epsilon-Greedy	≈ 690	2	Fixed exploration rate ($\epsilon = 0.01$)
Thompson Sampling	≈ 410	3	Bayesian probability matching
Random	≈ 80	4	Pure exploration (baseline)

- **Performance Gap:** UCB outperforms random selection by a factor of ≈ 9.5
- **Trend Analysis:** All algorithms show continued improvement even after 400K steps
- **Convergence:** No algorithm fully converges within the experiment timeframe

Dynamic Arm Addition: Adaptation Without Restarting

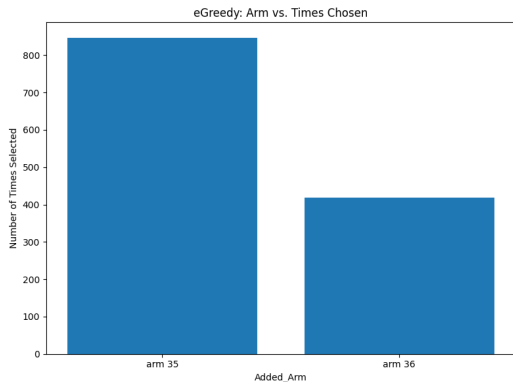


Figure: eGreedy: Arm vs. Times Chosen

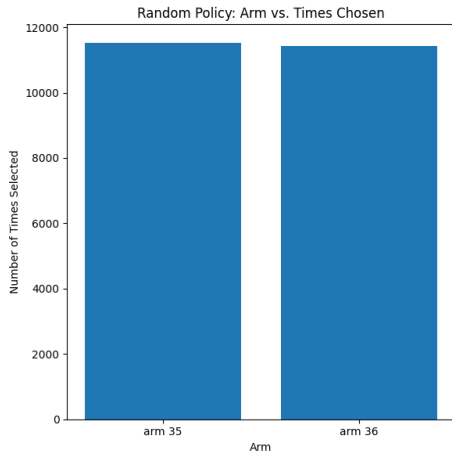


Figure: Random Policy: Arm vs. Times Chosen

Analysis: Dynamic Arm Addition

Random Policy Analysis:

- Equal selection of arms 35 & 36 ($\approx 11,500$ times each)
- Uniform exploration across all arms
- No preference despite late addition of these arms
- Represents pure exploration behavior

ϵ -Greedy Analysis:

- Significant preference for arm 35 (≈ 850 selections)
- Limited selection of arm 36 (≈ 420 selections)
- Demonstrates exploitation of better-performing arms
- Late-added arms selected less frequently

Implications

- **Algorithm Adaptability:** Bandit algorithms can handle dynamic arm addition without restarting
- **Exploration-Exploitation Tradeoff:** ϵ -Greedy focuses on high-reward arms while Random explores uniformly
- **Real-world Application:** Critical for recommender systems where new items are continuously added

Thank You!

Questions?
